

Struktúrák C++-ban: a struct

Jegyzet és feladatlap

Hogyan dolgozz ezzel a jegyzettel?

Ugyanúgy, mint a függvényes jegyzettel: egyedül is végig tudsz menni rajta.

- **Kis lépések.** Minden újdonságot előbb egy kész kódon próbálsz ki, és csak utána írsz sajátot.
- **Ellenőrzés.** A feladatok megadják, mit kell kapnod. Ha a programod ugyanazt írja ki, mehetsz tovább.
- **Hibakereső.** Ha hibaüzenetet kapsz, lapozz a jegyzet végére, és keresd meg a táblázatban.

A jelölések a szokásosak: **Próbáld ki**, **Tippelj**, **Feladat**. A nehézséget csillagok jelzik: ★ = alap, ★★ = közepes, ★★★ = nehezebb.

Ha elakadsz: olvasd el újra a feladat előtti mintakódot, nézd meg a Hibakeresőt, kérdezd meg a padtársadat, és csak ezután szólj a tanárodnak.

1. Összetartozó adatok egy egységben

A **struct** segítségével saját típust készíthetsz, amely több összetartozó adatot fog össze egyetlen változóban.

Próbáld ki - adatok szétszórva

Három diák nevét, életkorát és átlagát tároljuk. Eddigi tudásunkkal három külön vektor kell hozzá:

```
#include <iostream>
#include <string>
#include <vector>

int main() {
    std::vector<std::string> nevek = {"Anna", "Bence", "Csilla"};
    std::vector<int> korok = {17, 16, 18};
    std::vector<double> atlagok = {4.5, 3.8, 4.9};

    for (int i = 0; i < nevek.size(); i++) {
        std::cout << nevek[i] << " " << korok[i] << " " << atlagok[i] << std::endl;
    }
    return 0;
}
```

Futtasd, aztán vegyél fel egy negyedik diákot: Dani, 17 éves, átlaga 2.9. Hány sort kellett módosítanod? Mi történik, ha az egyik vektorból kifelejtet az új adatot?

A gond: Anna adatai három különböző helyen vannak. Semmi nem mutatja, hogy összetartoznak.

Próbáld ki - az első struct

```
#include <iostream>
#include <string>

struct Diak {
    std::string nev;
    int kor;
    double atlag;
};

int main() {
    Diak d;
    d.nev = "Anna";
    d.kor = 17;
    d.atlag = 4.5;

    std::cout << d.nev << " " << d.kor << " " << d.atlag << std::endl;
    return 0;
}
```

Nézzük meg a részeit:

- **struct Diak { ... };** - egy új típus leírása. A neve **Diak**; a saját típusok nevét nagy kezdőbetűvel szokás írni.
- A kapcsos zárójelek között a **mezők** állnak: mindegyiknek típusa és neve van, mint egy változónak.
- A záró kapcsos zárójel után **pontosvessző** áll. Ezt könnyű elfelejteni!
- **Diak d;** - létrehozunk egy **Diak** típusú változót. Ugyanúgy, ahogy az **int x;** egy **int** típusút.
- **d.nev** - a **d** változó **nev** mezője. A mezőket ponttal éred el.

A **struct** olyan, mint egy üres űrlap terve: megmondja, milyen rovatok vannak rajta. A **d** változó egy kitöltött űrlap.

Kísérlet - a pontosvessző

Töröld ki a **struct** záró kapcsos zárójel utáni pontosvesszőt. Olvasd el a hibaüzenetet, és keresd meg a Hibakeresőben. Utána írd vissza.

Kísérlet - hol legyen a struct?

Vágd ki a **struct Diak** leírását (mind az öt sorát), és illeszd be a **main** után. Lefordul? A szabály ugyanaz, mint a függvényeknél.

Figyelem! A **struct** leírását a program elejére írd: a **main** és minden olyan függvény elé, amely használja.

Feladatok

1.1. feladat ★☆☆ - Saját adatok

Gépeld be az első structos programot, és írd át benne az adatokat a sajátjaidra.

Ellenőrzés: egy sor jelenik meg a neveddel, az életkoroddal és egy átlaggal.

1.2. feladat ★☆☆ - Új mező

Vegyél fel a **Diak** structba egy negyedik mezőt: **std::string osztaly;**. Adj neki értéket a **main**-ben, és írd ki a többi adattal együtt.

Ellenőrzés: például **Anna 17 4.5 11.B**

1.3. feladat ★☆☆ - Két diák

Hozz létre a `main`-ben két `Diak` típusú változót (`d1` és `d2`), töltsd ki mindkettőt különböző adatokkal, és írd ki őket két sorba. Hányszor kellett leírnod a `struct` leírását?

Ellenőrzés: két sor, két különböző diák adataival.

1.4. feladat ★☆☆ - Könyv

Írj `Könyv` nevű `struct`ot három mezővel: `cim` és `szerzo` (szöveg), `oldalszam` (egész). Hozz létre egy könyvet, és írd ki az adatait.

Ellenőrzés: például `Egri csillagok (Gardonyi Geza), 560 oldal`

2. Létrehozás, módosítás, másolás

A `struct` mezői ugyanúgy viselkednek, mint a közönséges változók: értéket kaphatnak, beolvashatók, számolni lehet velük.

Próbáld ki - kezdőérték kapcsos zárójellel

A mezőket egyetlen sorban is kitöltheted. Az értékek abban a sorrendben kerülnek a mezőkbe, ahogy a mezők a `struct` leírásában állnak.

```
Diak d = {"Anna", 17, 4.5};

std::cout << d.nev << " " << d.kor << " " << d.atlag << std::endl;
```

Tippelj - számít a sorrend?

```
#include <iostream>

struct Datum {
    int ev;
    int honap;
    int nap;
};

int main() {
    Datum a = {2026, 10, 4};
    Datum b = {4, 10, 2026};

    std::cout << a.ev << " " << b.ev << std::endl;
    return 0;
}
```

A fordító nem tudja, mit jelentenek a számaid: a helyes sorrendre neked kell figyelned.

Próbáld ki - módosítás és beolvasás

```
Diak d = {"Anna", 17, 4.5};

d.kor = d.kor + 1;
d.atlag = 4.7;
std::cout << d.nev << " " << d.kor << " " << d.atlag << std::endl;

std::cin >> d.nev >> d.kor >> d.atlag;
std::cout << d.nev << " " << d.kor << " " << d.atlag << std::endl;
```

Tippelj - másolat vagy ugyanaz?

```
Diak d = {"Anna", 17, 4.5};
Diak masik = d;

masik.nev = "Bence";

std::cout << d.nev << " " << masik.nev << std::endl;
```

Az = jel az **összes** mezőt lemásolja. A **masik** ezután önálló változó: ha módosítod, a **d** nem változik.

Kísérlet - amit a struct nem tud

Próbáld ki ezt a három dolgot **egyesével**. Mindegyiknél olvasd el a hibaüzenetet vagy figyeld meg a kimenetet, és keresd meg a Hibakeresőben.

```
Diak d;
std::cout << d.kor << std::endl;
```

```
Diak d = {"Anna", 17, 4.5};
std::cout << d << std::endl;
```

```
Diak a = {"Anna", 17, 4.5};
Diak b = {"Anna", 17, 4.5};
if (a == b) {
    std::cout << "egyformak" << std::endl;
}
```

Figyelem! Egy structot nem lehet egyben kiírni és nem lehet == jellel összehasonlítani: mindkettőt mezőnként kell megtenni. Ha pedig nem adsz értéket a mezőknek, értelmetlen számok lehetnek bennük.

Feladatok

2.1. feladat ★☆☆ - Dátum

Írd meg a **Datum** structot (**ev**, **honap**, **nap**). Hozd létre a mai dátumot kapcsos zárójeles kezdőértékkel, és írd ki pontokkal elválasztva.

Ellenőrzés: például **2026.10.4.**

2.2. feladat ★☆☆ - Beolvasott diák

Olvasd be egy diák nevét (egy szó), életkorát és átlagát egy **Diak** típusú változóba, majd írd ki így:

Anna (17 éves) atlaga: 4.5

2.3. feladat ★☆☆ - Téglalap

Írj **Teglalap** structot két egész mezővel: **szelesseg** és **magassag**. Hozz létre egy 5 × 3-as téglalapot, és a **main**-ben számold ki a mezőiből a területét és a kerületét.

Ellenőrzés: terület 15, kerület 16.

2.4. feladat ★☆☆ - Ki az idősebb?

Hozz létre két diákot: {"Anna", 17, 4.5} és {"Csilla", 18, 4.9}. Írd ki annak a nevét, aki idősebb, majd annak a nevét, akinek jobb az átlaga.

Ellenőrzés: mindkét esetben **Csilla**. Cseréld ki Anna életkorát 19-re: az első válasz **Anna** lesz.

2.5. feladat ★★☆☆ - Időpont

Írj **Ido** structot (**ora**, **perc**). Hozz létre egy 7 óra 45 perces időpontot, és írd ki, hány perc telt el éjfél óta. Ezután növeld meg az időpontot 30 perccel úgy, hogy a perc 60 alatt maradjon.

Ellenőrzés: 465 perc. A növelés után az időpont 8 óra 15 perc.

3. Struct és függvény

Egy struct ugyanúgy lehet függvény paramétere vagy visszatérési értéke, mint egy szám. Így egyetlen paraméterben utazik az összes összetartozó adat.

Próbáld ki - struct paraméter

```
#include <iostream>
#include <string>

struct Diak {
    std::string nev;
    int kor;
    double atlag;
};

void kiir(Diak d) {
    std::cout << d.nev << " (" << d.kor << " éves) atlaga: " << d.atlag << std::endl;
}

bool kituno(Diak d) {
    return d.atlag >= 4.5;
}

int main() {
    Diak a = {"Anna", 17, 4.5};
    Diak b = {"Bence", 16, 3.8};

    kiir(a);
    kiir(b);

    if (kituno(a)) {
        std::cout << a.nev << " kituno." << std::endl;
    }
    return 0;
}
```

Gondold végig: struct nélkül a **kiir** függvénynek három paraméter kellett volna. És ha a diáknak tíz adata van?

Próbáld ki - függvény, amely structot ad vissza

```
Diak beolvas() {
    Diak d;
    std::cin >> d.nev >> d.kor >> d.atlag;
    return d;
}
```

A **main**-ben így használd: **Diak uj = beolvas();**, majd **kiir(uj);**

Tippelj - megváltozik-e?

```
void szulinap(Diak d) {
    d.kor = d.kor + 1;
}
```

A `main`-ben: `Diak a = {"Anna", 17, 4.5};`, aztán `szulinap(a);`, végül `kiir(a);`. Hány éves Anna a kiírásban?

Emlékezz a függvényes jegyzet 5. fejezetére: a paraméter másolat. Írd át a paramétert erre: `Diak& d`, és próbáld ki újra.

Feladatok

3.1. feladat ★☆☆ - Kiíró

Gépeld be a `kiir` függvényt, és írd ki vele három különböző diákot.

Ellenőrzés: három sor, például `Anna (17 éves) atlaga: 4.5`

3.2. feladat ★☆☆ - Terület és kerület

A 2.3. feladat `Teglalap` structjához írd két függvényt: `int terület(Teglalap t)` és `int kerület(Teglalap t)`.

Ellenőrzés: az 5×3 -as téglalapra `terület` $\rightarrow 15$, `kerület` $\rightarrow 16$.

3.3. feladat ★☆☆ - Nagykorú

Írd `bool nagykoru(Diak d)` függvényt.

Ellenőrzés: `{"Csilla", 18, 4.9}` $\rightarrow 1$, `{"Anna", 17, 4.5}` $\rightarrow 0$.

3.4. feladat ★☆☆ - Idősebb

Írd `Diak idosebb(Diak a, Diak b)` függvényt, amely a két diák közül az idősebbet adja vissza. Az eredményt a `kiir` függvénnyel írd ki.

Ellenőrzés: Anna (17) és Csilla (18) közül Csilla adatai jelennek meg.

3.5. feladat ★☆☆ - Percekben

A 2.5. feladat `Ido` structjához írd `int percben(Ido i)` függvényt, és egy másikat, amely visszafelé alakít: `Ido idobol(int perc)`.

Ellenőrzés: `percben({7, 45})` $\rightarrow 465$. Az `idobol(465)` eredményének `ora` mezője 7, `perc` mezője 45.

3.6. feladat ★☆☆ - Korábbi dátum

Írd `bool korabbi(Datum a, Datum b)` függvényt, amely megmondja, hogy az `a` dátum korábbi-e a `b`-nél.

Tipp: előbb az évet hasonlítsd össze; ha egyenlő, a hónapot; ha az is egyenlő, a napot.

Ellenőrzés: `{2026, 10, 4}` és `{2026, 12, 1}` $\rightarrow 1$; `{2026, 10, 4}` és `{2025, 12, 31}` $\rightarrow 0$; `{2026, 10, 4}` és `{2026, 10, 4}` $\rightarrow 0$.

4. Függvény a structon belül

Azt a függvényt, amely egy struct adataival dolgozik, a struct belsejébe is írhatod. Ezt **tagfüggvénynek** hívjuk.

Próbáld ki

```
#include <iostream>

struct Teglalap {
    int szelesseg;
    int magassag;

    int terület() {
        return szelesseg * magassag;
    }
};

int main() {
    Teglalap t = {5, 3};

    std::cout << t.terület() << std::endl;
    return 0;
}
```

Hasonlítsd össze a 3.2. feladat megoldásával. Három dolog változott:

- A függvény a struct kapcsos zárójelei közé került.
- Nincs paramétere: a mezőket (**szelesseg**, **magassag**) közvetlenül látja.
- A hívás ponttal történik, mint a mezők elérése: **t.terület()**. A függvény a pont előtt álló változó adataival dolgozik.

Ismerős ez a forma? A **szo.length()** és a **v.size()** is tagfüggvény volt.

Tippelj - kinek az adataival számol?

```
Teglalap a = {5, 3};
Teglalap b = {10, 10};

std::cout << a.terület() << " " << b.terület() << std::endl;
```

Próbáld ki - tagfüggvény paraméterrel, és amelyik módosít

Bővítsd a **Teglalap** structot ezzel a két tagfüggvénnyel:

```
bool nagyobb(Teglalap masik) {
    return terület() > masik.terület();
}

void nagyit(int szorzo) {
    szelesseg = szelesseg * szorzo;
    magassag = magassag * szorzo;
}
```

A **main**-ben próbáld ki:

```
Teglalap a = {5, 3};
Teglalap b = {4, 4};

std::cout << a.nagyobb(b) << std::endl;

a.nagyit(2);
std::cout << a.szelesseg << " x " << a.magassag << std::endl;
std::cout << a.terulet() << std::endl;
```

Figyeld meg:

- A tagfüggvény meghívhat egy másik tagfüggvényt (**terulet()**), pont nélkül: ilyenkor az a saját adataival számol.
- A **nagyit** megváltoztatja a mezőket, és ez a változás megmarad. Itt nincs szükség **&** jelre: a tagfüggvény magán az eredeti változón dolgozik.

Kísérlet - hibás hívások

Próbáld ki egyesével a **main**-ben, és keresd meg a hibaüzenetet a Hibakeresőben:

```
std::cout << terulet() << std::endl;
std::cout << t.terulet << std::endl;
```

Tipp: Mikor írd a függvényt a structba? Ha egyetlen struct adataiból számol vagy azokat módosítja (terület, kiírás, növelés), akkor legyen tagfüggvény. Mindkét megoldás helyes; válaszd azt, amelyikkel olvashatóbb a programod.

Feladatok

4.1. feladat ★☆☆ - Kerület

Írj a **Teglalap** structba **int kerulet()** tagfüggvényt.

Ellenőrzés: az {5, 3} téglalapra **t.kerulet()** → 16.

4.2. feladat ★☆☆ - Négyzet-e?

Írj a **Teglalap** structba **bool negyzet()** tagfüggvényt.

Ellenőrzés: {4, 4} → 1, {5, 3} → 0.

4.3. feladat ★☆☆ - Kiíró tagfüggvény

Írd át a 3. fejezet **kiir** függvényét a **Diak** struct tagfüggvényévé. A hívás ezután: **a.kiir()**;

Ellenőrzés: Anna (17 éves) atlaga: 4.5

4.4. feladat ★☆☆ - Kitűnő, nagykorú

Írj a **Diak** structba két tagfüggvényt: **bool kituno()** (az átlag legalább 4.5) és **bool nagykoru()**.

Ellenőrzés: {"Csilla", 18, 4.9} → mindkettő 1; {"Bence", 16, 3.8} → mindkettő 0.

4.5. feladat ★☆☆ - Időpont tagfüggvényekkel

Írd meg az **Ido** structot (**ora**, **perc**) két tagfüggvénnyel: az **int percben()** az éjfél óta eltelt percekot adja vissza, a **void kiir()** pedig kiírja az időpontot.

Ellenőrzés: a {7, 45} időpontra **percben()** → 465, a **kiir()** kimenete 7:45.

4.6. feladat ★★☆☆ - Bankszámla

Írj `Szaml` structot két mezővel: `tulajdonos` (szöveg) és `egyenleg` (egész). Tagfüggvényei: `void befizet(int osszeg)` megnöveli az egyenleget; `bool kivesz(int osszeg)` csak akkor csökkenti, ha van elég pénz a számlán, és visszaadja, hogy sikerült-e.

Ellenőrzés: a számla 1000-ról indul. `befizet(500)` után az egyenleg 1500. `kivesz(2000)` → 0, az egyenleg marad 1500. `kivesz(700)` → 1, az egyenleg 800.

4.7. feladat ★★☆☆ - Tört

Írj `Tort` structot (`samlalo`, `nevezo`). Tagfüggvényei: `double ertek()` a tört értékét adja vissza tizedes törtként; `void kiir()` kiírja 3/4 formában; `void egyszerusit()` a lehető legegyszerűbb alakra hozza.

Tipp: keresd meg a legnagyobb számot, amely a számlálót és a nevezőt is osztja.

Ellenőrzés: a {6, 8} törtre `ertek()` → 0.75; `egyszerusit()` után a `kiir()` kimenete 3/4.

5. Structok vektora

Egy vektor elemei structok is lehetnek. Így sok diák, termék vagy versenyző összes adatát egyetlen vektorban tárolod – az érettségi feladatokban szinte mindig erre van szükség.

Próbáld ki

```
#include <iostream>
#include <string>
#include <vector>

struct Diak {
    std::string nev;
    int kor;
    double atlag;
};

int main() {
    std::vector<Diak> diakok = {
        {"Anna", 17, 4.5},
        {"Bence", 16, 3.8},
        {"Csilla", 18, 4.9},
        {"Dani", 17, 2.9},
        {"Emma", 16, 4.2}
    };

    for (int i = 0; i < diakok.size(); i++) {
        std::cout << diakok[i].nev << " " << diakok[i].atalag << std::endl;
    }
    return 0;
}
```

Olvasd ki balról jobbra: `diakok[i].nev` – a `diakok` vektor `i` indexű eleme (egy `Diak`), annak pedig a `nev` mezője.

Hasonlítsd össze az 1. fejezet három vektorával. Most hány sort kell írnod, ha új diák érkezik? És ha új mező kell?

Tippelj

```
std::cout << diakok[2].nev << std::endl;
std::cout << diakok[0].kor + diakok[1].kor << std::endl;
std::cout << diakok[diakok.size() - 1].nev << std::endl;
```

Próbáld ki - új elem hozzáadása

```
Diak uj = {"Feri", 18, 3.5};
diakok.push_back(uj);

std::cout << diakok.size() << std::endl;
```

Próbáld ki - a jól ismert minták, egy mezőre

A vektoros jegyzet minden mintája működik, csak most egy **mezőt** vizsgálsz. Megszámlálás: hány diák átlaga legalább 4.0?

```
int db = 0;
for (int i = 0; i < diakok.size(); i++) {
    if (diakok[i].atlag >= 4.0) {
        db++;
    }
}
std::cout << db << std::endl;
```

Maximumkiválasztás: ki a legidősebb? Az indexet jegyezzük meg, mert a végén a nevére vagyunk kíváncsiak, nem az életkorára.

```
int maxIndex = 0;
for (int i = 1; i < diakok.size(); i++) {
    if (diakok[i].kor > diakok[maxIndex].kor) {
        maxIndex = i;
    }
}
std::cout << diakok[maxIndex].nev << std::endl;
```

Feladatok

Mindegyik feladatban a fenti öt diák adataival dolgozz.

5.1. feladat ★☆☆ - Névsor

Írd ki minden diák összes adatát, soronként egy diákot. Ha megírtad a 4.3. feladat **kiir** tagfüggvényét, használd: **diakok[i].kiir();**

Ellenőrzés: öt sor, az első **Anna (17 éves) átlaga: 4.5**

5.2. feladat ★☆☆ - Az átlagok átlaga

Számold ki az öt diák átlagának átlagát.

Ellenőrzés: 4.06.

5.3. feladat ★☆☆ - Tizenhét évesek

Hány diák 17 éves? Írd ki a nevüket is.

Ellenőrzés: 2 diák, Anna és Dani.

5.4. feladat ★☆☆ - A legjobb tanuló

Írd ki a legjobb átlagú diák nevét és átlagát.

Ellenőrzés: **Csilla 4.9**

5.5. feladat ★☆☆ - Keresés név alapján

Olvass be egy nevet, és írd ki az illető átlagát. Ha nincs ilyen nevű diák, ezt írd ki: **Nincs ilyen diak.**

Ellenőrzés: **Dani** → 2.9; **Zoli** → **Nincs ilyen diak.**

5.6. feladat ★★☆☆ - Kiválogatás

Válogasd ki egy új `std::vector<Diak>` vektorba azokat, akiknek az átlaga legalább 4.0. Írd ki az új vektor méretét és a neveket.

Ellenőrzés: 3 diák: Anna, Csilla, Emma.

5.7. feladat ★★☆☆ - Beolvasás

Kérdezd meg, hány diák lesz. Olvasd be mindegyik nevét, életkorát és átlagát, és tedd őket egy vektorba. A végén írd ki a legfiatalabb diák nevét.

Ellenőrzés: ha a fenti öt diák adatait írod be, az eredmény **Bence**.

5.8. feladat ★★☆☆ - Függvényekkel

Írj két függvényt: a `double atlagokAtlaga(std::vector<Diak> v)` az átlagok átlagát, a `Diak legjobb(std::vector<Diak> v)` a legjobb átlagú diákot adja vissza. A `main` csak meghívja őket, és kiírja az eredményt.

Ellenőrzés: 4.06 és Csilla adatai.

6. Egy sorból egy struct

A feladatokban az adatok soronként érkeznek, egy sorban egy diák (termék, versenyző) összes adata. Egy függvény a sorból structot készít.

Próbáld ki

A sor feldarabolását a sztringes jegyzet 9. fejezetéből ismered. Most a darabok egy struct mezőibe kerülnek:

```
#include <iostream>
#include <sstream>
#include <string>

struct Diak {
    std::string nev;
    int kor;
    double atlag;
};

Diak feldolgoz(std::string sor) {
    std::stringstream ss(sor);
    std::string korSzoveg, atlagSzoveg;

    Diak d;
    std::getline(ss, d.nev, ';');
    std::getline(ss, korSzoveg, ';');
    std::getline(ss, atlagSzoveg, ';');
    d.kor = std::stoi(korSzoveg);
    d.atlag = std::stod(atlagSzoveg);
    return d;
}

int main() {
    Diak d = feldolgoz("Kiss Anna;17;4.5");

    std::cout << d.nev << " jovore " << d.kor + 1 << " éves lesz." << std::endl;
    return 0;
}
```

Próbáld ki - sok sor, structok vektora

Cseréld le a `main` függvényt erre. A vektorhoz kell az `#include <vector>` sor is.

```
int main() {
    int db;
    std::cin >> db;
    std::cin.ignore();

    std::vector<Diak> diakok;
    for (int i = 0; i < db; i++) {
        std::string sor;
        std::getline(std::cin, sor);
        diakok.push_back(feldolgoz(sor));
    }

    std::cout << diakok.size() << " diak adatai beolvasva." << std::endl;
    return 0;
}
```

Ez a három lépés (sor beolvasása, structtá alakítása, vektorba tétele) szinte minden nagyobb feladat eleje. Ha ez megvan, a kérdésekre az 5. fejezet mintáival válaszolsz.

Feladatok

6.1. feladat ★☆☆ - Ellenőrző kiírás

Bővítsd a fenti programot: a beolvasás után írd ki minden diák nevét és átlagát.

Ellenőrzés: írd be ezt a három sort a darabszám (3) után, és nézd meg, hogy mind megjelenik-e:

```
Kiss Anna;17;4.5
Nagy Bence;16;3.8
Toth Csilla;18;4.9
```

6.2. feladat ★☆☆ - Termék

Írj `Termek` structot: `nev` (szöveg), `ar` és `darab` (egész). Legyen egy `int ertek()` tagfüggvénye, amely az ár és a darabszám szorzatát adja vissza. Írj hozzá `Termek feldolgoz(std::string sor)` függvényt a `nev;ar;darab` formájú sorokhoz.

Ellenőrzés: a `fuzet;350;12` sorból készült termékre `ertek()` → 4200.

6.3. feladat ★☆☆ - Raktár

Olvass be négy terméket egy vektorba, és válaszolj a kérdésekre: mennyi a teljes készlet értéke? Melyik a legdrágább termék? Melyik termék készlete ér a legtöbbet?

```
fuzet;350;12
toll;180;30
radir;120;8
vonalzo;420;5
```

Ellenőrzés: a teljes érték 12660; a legdrágább a `vonalzo`; a legtöbbet a `toll` készlete éri (5400).

6.4. feladat ★★★ - Szóközzel tagolt sor

Írj `Diak feldolgoz2(std::string sor)` függvényt olyan sorokhoz, amelyekben szóköz az elválasztó, és a név egyetlen szó: `Anna 17 4.5`. **Tipp:** itt elég a `ss >> d.nev >> d.kor >> d.atlag;`

Ellenőrzés: az `Anna 17 4.5` sorból készült diák `kor` mezője 17, `atlag` mezője 4.5.

7. Rendezés egy mező szerint

A `std::sort` nem tudja magától, melyik mező szerint rendezzen. Egy összehasonlító függvénnyel mondd meg neki.

Kísérlet - rendezés segítség nélkül

Az 5. fejezet programjában próbáld meg rendezni a diákokat. Kell hozzá az `#include <algorithm>` sor.

```
std::sort(diakok.begin(), diakok.end());
```

Nagyon hosszú hibaüzenetet kapsz. A lényege: a fordító nem tudja, mit jelent az, hogy az egyik diák „kisebb”, mint a másik.

Próbáld ki - összehasonlító függvény

Írd ezt a függvényt a `main` elé:

```
bool atlagSzerint(Diak a, Diak b) {  
    return a.atlag < b.atlag;  
}
```

A rendezésnél harmadik adatként add meg a függvény **nevét** (zárójelek nélkül):

```
std::sort(diakok.begin(), diakok.end(), atlagSzerint);  
  
for (int i = 0; i < diakok.size(); i++) {  
    std::cout << diakok[i].nev << " " << diakok[i].atlag << std::endl;  
}
```

Az összehasonlító függvény arra válaszol: „az `a` előbb álljon-e, mint a `b`?” Írd át benne a `<` jelet `>` jelre. Mi változik?

Feladatok

7.1. feladat ★☆☆ - Névsor

Rendezd az öt diákot név szerint ábécérendbe, és írd ki a neveket.

Ellenőrzés: Anna, Bence, Csilla, Dani, Emma. (Ez már eleve így volt; hogy lásd a rendezést, keverd össze a diákokat a vektor megadásakor.)

7.2. feladat ★★☆☆ - Rangsor

Írd ki a diákokat átlag szerint csökkenő sorrendben, sorszámozva.

Ellenőrzés: 1. Csilla 4.9, 2. Anna 4.5, 3. Emma 4.2, 4. Bence 3.8, 5. Dani 2.9

7.3. feladat ★★★ - Kor, azon belül név

Rendezd a diákokat életkor szerint növekvő sorrendbe; az egyidősek névsorban kövessék egymást.

Tipp: ha a két életkor különbözik, az dönt; ha egyenlő, a név.

Ellenőrzés: Bence, Emma, Anna, Dani, Csilla.

8. Structok egymásban, és egy nagy feladat

Egy struct mezője lehet másik struct vagy vektor is. Így bonyolultabb adatokat is egy egységbe tudsz szervezni.

Próbáld ki - struct a structban

```
#include <iostream>
#include <string>

struct Datum {
    int ev;
    int honap;
    int nap;
};

struct Tanulo {
    std::string nev;
    Datum szuletes;
};

int main() {
    Tanulo t = {"Anna", {2009, 5, 17}};

    std::cout << t.nev << " szuletesi eve: " << t.szuletes.ev << std::endl;
    return 0;
}
```

Olvasd ki balról jobbra: `t.szuletes.ev` - a `t` tanuló `szuletes` mezője (egy `Datum`), annak pedig az `ev` mezője.

Próbáld ki - vektor a structban

```
#include <iostream>
#include <string>
#include <vector>

struct Naplo {
    std::string nev;
    std::vector<int> jegyek;

    double atlag() {
        double osszeg = 0;
        for (int i = 0; i < jegyek.size(); i++) {
            osszeg = osszeg + jegyek[i];
        }
        return osszeg / jegyek.size();
    }
};

int main() {
    Naplo n = {"Anna", {5, 3, 4, 5, 2}};
    std::cout << n.nev << " atlaga: " << n.atlag() << std::endl;

    n.jegyek.push_back(5);
    std::cout << n.nev << " atlaga: " << n.atlag() << std::endl;
    return 0;
}
```

Most már nem kell az átlagot külön mezőben tárolni: a struct a jegyekből bármikor kiszámolja.

Feladatok

8.1. feladat ★★☆☆ - Életkor

Írj a `Tanulo` structba `int életkor(int ev)` tagfüggvényt, amely megadja, hány éves lesz a tanuló a megadott év végén.

Ellenőrzés: a 2009. május 17-én született tanulóra `t.életkor(2026)` → 17.

8.2. feladat ★★☆☆ - Napló

Bővítsd a `Naplo` structot két tagfüggvénnyel: az `int legjobb()` a legjobb jegyet, az `int hanyOtos()` az ötösök számát adja vissza.

Ellenőrzés: az `{5, 3, 4, 5, 2}` jegyekre `legjobb()` → 5, `hanyOtos()` → 2.

8.3. feladat ★★☆☆ - Osztálynapló

Hozz létre egy `std::vector<Naplo>` vektort három tanulóval: Anna jegyei 5, 3, 4, 5, 2; Bence jegyei 4, 4, 3; Csilla jegyei 5, 5, 4, 5. Írd ki mindenki nevét az átlagával, majd a legjobb átlagú tanuló nevét.

Ellenőrzés: Anna 3.8, Bence 3.66667, Csilla 4.75; a legjobb Csilla.

8.4. feladat ★★☆☆ - Futóverseny

Ebben a feladatban mindent felhasználsz, amit tanultál. Egy futóverseny eredményei soronként érkeznek, `rajtszam;nev;perc;masodperc` formában:

```
12;Kiss Anna;24;35
7;Nagy Bence;22;58
31;Toth Csilla;23;10
4;Szabo Dani;26;2
18;Varga Emma;22;41
```

Írj `Futo` structot a négy mezővel és egy `int osszMasodperc()` tagfüggvénnyel. Írj

`Futo feldolgoz(std::string sor)` függvényt. Olvasd be az öt sort egy vektorba (előbb a darabszámot), aztán válaszolj:

- Ki nyerte a versenyt, és mennyi volt az ideje?
- Hány futó ért be 25 percen belül?
- Mennyi az átlagos idő másodpercben?
- Mi a végeredmény? Írd ki a futókat a beérkezés sorrendjében.

Ellenőrzés: a győztes Varga Emma (22 perc 41 másodperc). 25 percen belül 4 futó ért be. Az átlagos idő 1433.2 másodperc. A sorrend: Varga Emma, Nagy Bence, Toth Csilla, Kiss Anna, Szabo Dani.

Építsd lépésenként: előbb csak a struct és a `feldolgoz` függvény egyetlen sorral kipróbálva, aztán a beolvasás, és csak utána a kérdések, egyenként.

Hibakereső: mit jelent a hibaüzenet?

Keresd meg a táblázatban azt a sort, amely hasonlít arra, amit látsz. A hibaüzenet pontos szövege fordítónként kicsit eltérhet.

Amit látsz	Mit jelent?	Mit tegyél?
<code>expected ';' after struct definition</code>	Lemaradt a pontosvessző a <code>struct</code> záró kapcsos zárójele után.	Írd oda: <code>};</code>
<code>'Diak' was not declared in this scope</code> vagy <code>'Diak' does not name a type</code>	A fordító nem ismeri ezt a típust.	A <code>struct</code> leírása a használata előtt van? Jól írtad a nevét (nagybetű)?
<code>'struct Diak' has no member named 'eletkor'</code>	Ilyen nevű mezője nincs a structnak.	Nézd meg a <code>struct</code> leírásában, pontosan hogy hívják a mezőt.
<code>'class std::vector<Diak>' has no member named 'nev'</code>	A vektornak nincs ilyen mezője, csak egy-egy elemének.	Kimaradt az index: <code>diakok[i].nev</code>
<code>no match for 'operator<<'</code>	Egy egész structot próbálsz kiírni.	Írd ki mezőnként, vagy írd rá kiíró függvényt.
<code>no match for 'operator=='</code>	Két structot próbálsz <code>==</code> jellel összehasonlítani.	Hasonlítsd össze a mezőket egyenként.
Nagyon hosszú hiba a <code>std::sort</code> sorában (<code>no match for 'operator<'</code>)	A rendezés nem tudja, melyik mező szerint rendezzen.	Adj meg összehasonlító függvényt (7. fejezet).
<code>could not convert, narrowing conversion</code> vagy <code>too many initializers</code>	A kapcsos zárójeles kezdőértékek nem illenek a mezőkhöz.	Ellenőrizd az értékek sorrendjét, típusát és darabszámát.
<code>'terulet' was not declared in this scope</code>	Egy tagfüggvényt pont nélkül hívtál meg.	Írd elé a változót: <code>t.terulet()</code>
<code>invalid use of non-static member function</code>	Lemaradt a zárójel a tagfüggvény hívásáról.	<code>t.terulet</code> helyett <code>t.terulet()</code>
Nincs hiba, de értelmetlen számok jelennek meg	Nem adtál értéket a mezőknek.	Adj kezdőértéket: <code>Diak d = {"Anna", 17, 4.5};</code>
A függvény után a struct nem változott meg	A függvény csak másolatot kapott.	A paraméter legyen <code>Diak& d</code> , vagy írd meg tagfüggvényként.
A beolvasásnál az első sor üres	A darabszám után bent maradt a sorvége.	Kell a <code>std::cin.ignore();</code> a <code>std::getline</code> előtt.

Összefoglaló táblázat (puska)

Minta	Jelentés
<code>struct Diak { std::string nev; int kor; };</code>	új típus leírása két mezővel (a végén pontosvessző!)
<code>Diak d;</code>	Diak típusú változó létrehozása
<code>Diak d = {"Anna", 17};</code>	létrehozás kezdőértékekkel, a mezők sorrendjében
<code>d.nev</code>	a d változó nev mezője
<code>Diak masik = d;</code>	másolat az összes mezőről
<code>void kiir(Diak d) { ... }</code>	függvény struct paraméterrel
<code>Diak beolvas() { ... }</code>	függvény, amely structot ad vissza
<code>int terület() { return a * b; }</code>	tagfüggvény a structon belül: a mezőket közvetlenül látja
<code>t.terulet()</code>	tagfüggvény hívása a t változó adataival
<code>std::vector<Diak> diakok;</code>	structok vektora
<code>diakok[i].nev</code>	az i indexű elem nev mezője
<code>diakok.push_back(d);</code>	új struct a vektor végére
<code>std::sort(v.begin(), v.end(), fuggveny);</code>	rendezés összehasonlító függvénnyel

Figyelem! Három dolog, amit soha ne felejts el: a **struct** leírása után pontosvessző áll; a mezőket ponttal éred el; a kapcsos zárójeles kezdőértékek sorrendje a mezők sorrendjét követi.